

Repo Github : <https://github.com/justeowen/UE03-Donn-es-et-Analytics>

UE03 Données et Analytics : Observabilité d'un jeu multijoueur web

Introduction et Architecture:

Le studio exploite en bêta un jeu de tir, mais ne dispose ni de pipeline, ni de centralisation des logs, ni de métriques. Les joueurs se plaignent de saccades depuis jeudi soir, et la direction se demande si certains abusent du jeu. La direction m'a donc contacté pour l'aider à répondre à ces deux questions.

Pour y répondre, j'ai d'abord automatisé la construction et le déploiement de Docker. Pour ensuite centraliser les logs dans Loki et les métriques dans Prometheus.

L'ensemble se lance avec une seule commande, `docker compose up`.

```
owen@LAPTOP-BSBCEN0U:~/hetic/rattrapage/UE03-Donn-es-et-Analytics$ docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
je03-donn-es-et-analytics-alloy-1	grafana/alloy:v1.8.0	"/bin/alloy run /etc_"	alloy	13 minutes ago	Up 13 minutes	
je03-donn-es-et-analytics-app-1	ue03-donn-es-et-analytics-app	"docker-entrpoint.s_"	app	13 minutes ago	Up 13 minutes	0.0.0.0:8080->8080/t
je03-donn-es-et-analytics-grafana-1	grafana/grafana:11.6.0	"/run.sh"	grafana	13 minutes ago	Up 13 minutes	0.0.0.0:3000->3000/t
je03-donn-es-et-analytics-loadgen-1	ue03-donn-es-et-analytics-loadgen	"docker-entrpoint.s_"	loadgen	13 minutes ago	Up 13 minutes	8080/tcp
je03-donn-es-et-analytics-loki-1	grafana/loki:3.4.2	"/usr/bin/loki -conf_"	loki	13 minutes ago	Up 13 minutes	3100/tcp
je03-donn-es-et-analytics-prometheus-1	prom/prometheus:v3.2.1	"/bin/prometheus --c_"	prometheus	13 minutes ago	Up 13 minutes	0.0.0.0:9090->9090/t

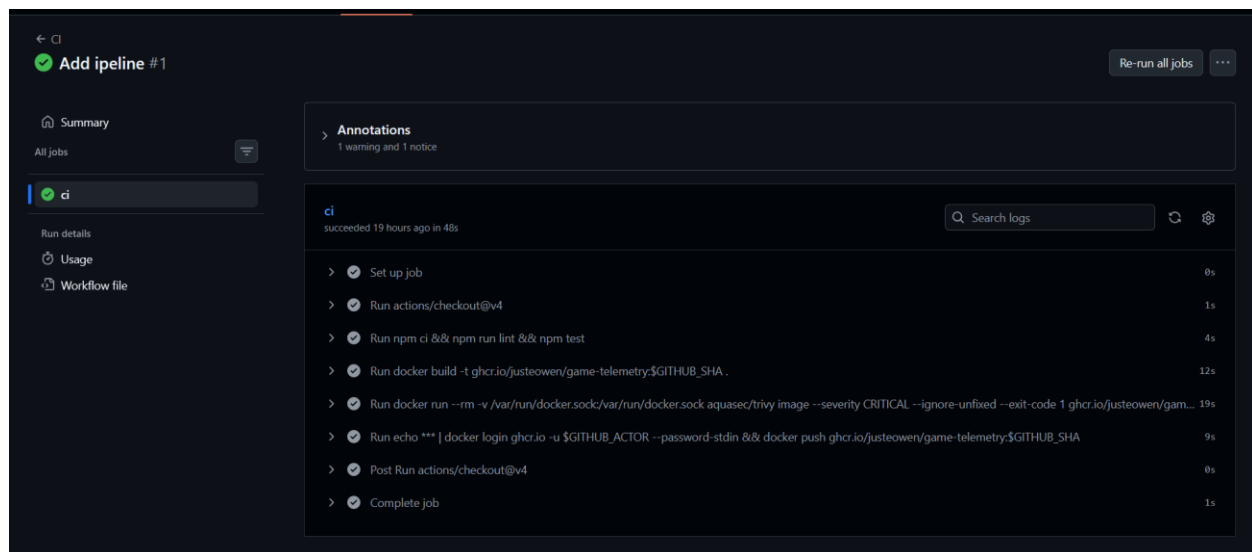
Partie A : CI/CD

Il y a 2 étapes dans les images une première installe les dépendances et la seconde ne garde que le résultat.

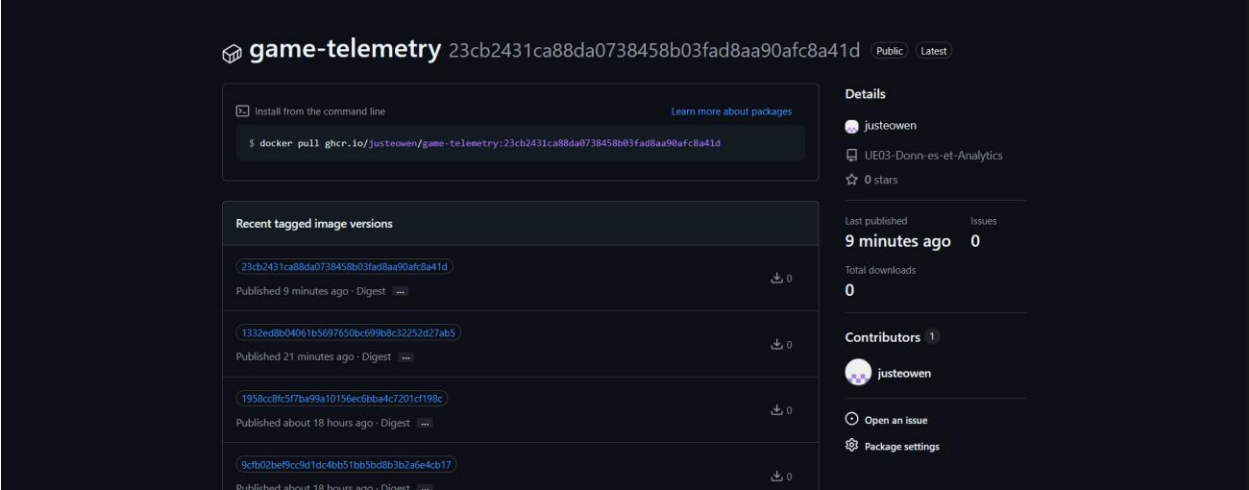
J'ai choisi Node 22 car Node 20 n'a plus de correctifs de sécurité depuis avril 2026.

Avec node:22-alpine l'image est légère et le service tourne avec l'utilisateur node plutôt que root.

À chaque push, GitHub Actions lance : lint et tests, build de l'image, scan Trivy, puis publiesur GHCR. Si une étape échoue, la suite est annulée.



Les 4 étapes s'exécutent en moins d'une minute : tests (4 s), build (12 s), scan Trivy (19 s) et publication (9 s)



Chaque version publiée porte le SHA du commit ce qui permet de remonter au code source.

Partie B : Logs et qualité des données

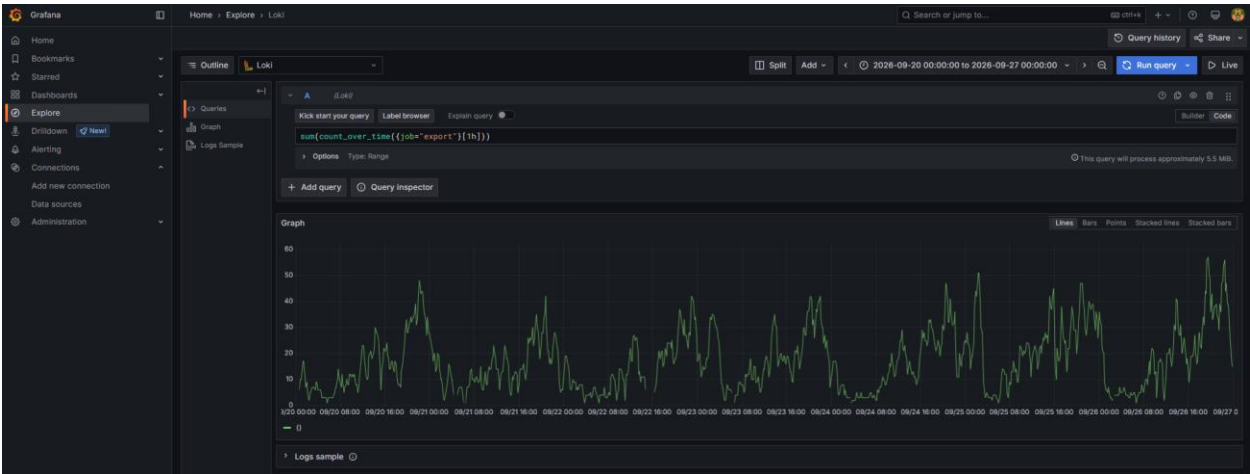
Les logs sont collecté par Grafana Alloy, puis stockés dans Loki.

```
wen@LAPTOP-BSBCEN0U:~/hetic/ratrapage/UE03-Donn-es-et-Analytics$ docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS        PORTS
je03-donn-es-et-analytics-alloy-1   grafana/alloy:v1.8.0               "/bin/alloy run /etc..."           alloy      13 minutes ago Up 13 minutes 0.0.0.0:8080->8080/tcp
je03-donn-es-et-analytics-app-1     ue03-donn-es-et-analytics-app      "docker-entrypoint.s..."           app        13 minutes ago Up 13 minutes 0.0.0.0:3000->3000/tcp
je03-donn-es-et-analytics-grafana-1 grafana/grafana:11.6.0             "/run.sh"                             grafana     13 minutes ago Up 13 minutes 0.0.0.0:3000->3000/tcp
je03-donn-es-et-analytics-loadgen-1 ue03-donn-es-et-analytics-loadgen  "docker-entrypoint.s..."           loadgen    13 minutes ago Up 13 minutes 8080/tcp
je03-donn-es-et-analytics-loki-1    grafana/loki:3.4.2                 "/usr/bin/loki -conf..."           loki       13 minutes ago Up 13 minutes 3100/tcp
je03-donn-es-et-analytics-prometheus-1 prom/prometheus:v3.2.1            "/bin/prometheus --c..."           prometheus 13 minutes ago Up 13 minutes 0.0.0.0:9090->9090/tcp
```

Constat	Chiffre
Volumétrie	264 344 lignes, 2 944 événements (2 148 rapports + 796 parties)
Format	Un événement s'étale sur plusieurs lignes (en-tête + résumé + JSON)
Horodatage	Date américaine en 12 h (12 AM = minuit), heure de Paris sans fuseau
Ordre	Fichier du plus récent au plus ancien
Doublons	55 blocs identiques
Schéma	25 rapports en version 2 (rttMs déplacé)

Incohérences	70 rapports avec des FPS impossibles (> 144)
--------------	--

L'export historique est lui aussi fait par Alloy, qui regroupe les lignes de chaque événements. Il va ensuite convertir la date et l'heure en UTC et ne garde que le JSON.



On observe un cycle de une semaine avec une grosse activité le soir.

Partie C : Métriques

Les métriques servent à détecter un problème : elles sont légères et faites pour les alertes. Les logs servent à comprendre : ils gardent le détail de chaque événements.

Prometheus interroge le service toutes les minutes.

C3 : Métriques ou logs ?

	Métriques (Prometheus)	Logs (Loki)
Contenu	Des chiffres : débit, erreurs, latence, parties en cours, rapports par version	Le détail de chaque événement : rapport complet, écran, navigateur, joueur
Rôle	Détecter un problème (alertes, tendances)	Comprendre pourquoi (enquête)

Coût	Faible	élevé
------	--------	-------

Partie D : Dashboards

Il y a trois dashboards provisionnés sous forme de code : ils sont dans le dossier dashboards/ et /Grafana les charge automatiquement au démarrage. Chaque panneau a pour titre la question à laquelle il répond.

- **Santé du service** : combien de requêtes par seconde reçoit l'API ?
Quelle part des requêtes est en erreur ? En combien de temps répondent 95 % des requêtes ?



- **Performance côté joueur** : quelles causes de saccades sont signalées ? Quelle version provoque les saccades d'overlay ?
Quels écrans sont touchés
- **Activité de jeu et intégrité des parties** : Combien de parties sont en cours ? Quels clients envoient des rapports impossibles ?

E. Détection de comportements

E1 : typologie

En comparant les champs des rapports, on peut identifier 5 types de saccades, chacun reconnaissable à une signature :

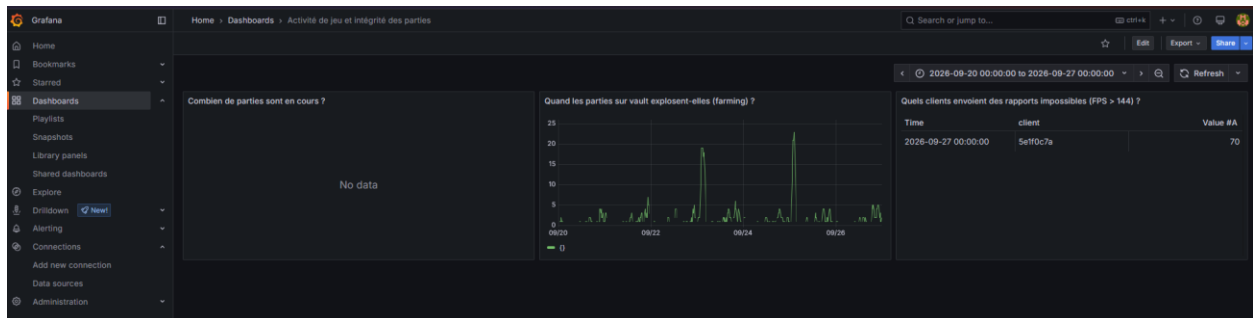
Type	Signature	Nombre
generic	rendu ×2 à ×6 (machine lente)	1 119
shader	compilation de shaders, début de partie	459
overlay	renderOverlay > 200 ms	262
hidden	onglet en arrière-plan	158
network	reason = network, RTT élevé	92

Chaque signature correspond à une requête LogQL, ce qui classe automatiquement les rapports



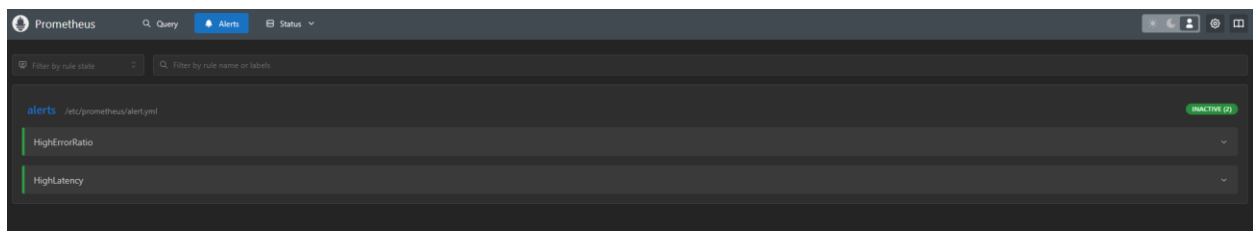
E2 : comportements anormaux

- Régression : avec la version beta-20260924-3, les saccades overlay passent de 0,8 % à 29 % des rapports.
- Population : 39 des 40 joueurs avec un écran d'au moins 2560 px et le bloom activé sont touchés.
- Phénomène récurrent : des dégradations réseau la nuit, vers 3 h .
- Farming : 83 parties de moins de 2 minutes, toutes sur vault avec un score de 3-0, sur 2 nuits .
- Données falsifiées : le client 5e1f0c7a a envoyé 70 rapports avec des FPS impossibles .



E3 : Alertes

Alerte	Seuil	Pourquoi
HighErrorRatio	> 10 % pendant 5 min	Il y a 3 % d'erreurs normales (le générateur envoie des rapports invalides)
HighLatency	p95 > 100 ms pendant 5 min	Les rafales courtes sont ignorées
OverlayRegression	> 5 saccades overlay en 10 min	Avant la régression, on en comptait environ 2 par jour



Les deux alertes Prometheus sont inactives en fonctionnement normal : il n'y a pas de faux positifs. En contrepartie, un problème qui dure moins de 5 minutes ne serait pas détecté (faux négatif).

F. Postmortem

Q1 : « Des saccades depuis jeudi soir. Pourquoi, et qui est touché ? »

Depuis le jeudi 24 septembre au soir, les joueurs équipés d'un grand écran avec le bloom activé subissent des saccades, causées par la version beta-20260924-3.

Heure (Paris)	Événement
Jeu. 24/09, 09:35	Déploiement de beta-20260924-3
Jeu. 24/09, 20:16	1 ^{re} saccade overlay
Ven. 25/09	114 saccades overlay
Sam. 26/09	124 saccades, toujours présentes dans les versions 26-5 et 26-6

- **Cause** : l'overlay passe de 3 ms à 220-620 ms par image quand le bloom est actif sur un grand écran.
- **Population** : 39 des 40 joueurs avec un écran d'au moins 2560 px et le bloom activé.
- **Actions correctives** : corriger l'overlay ou revenir à la version 22-1, et désactiver le bloom par défaut sur les grands écrans en attendant.
- **Actions préventives** : l'alerte OverlayRegression, et un suivi des rapports par version après chaque déploiement.

Q2 : « Est-ce que quelqu'un abuse du jeu ? »

Oui, deux abus ont été détectés :

- **Farming** : 83 parties très courtes sur vault. Seules 17 ont été mises en quarantaine.
- **Fausses Données** : le joueur 5e1f0c7a a envoyé 70 rapports.

Actions: Refuser les rapports avec des FPS élevé. Pourquoi pas ban le joueur temporairement

Conclusion. Grâce à ce travail, il est plus facile de détecter et d'expliquer les incidents, du déploiement jusqu'aux alertes.